Experiment No-14 : Alpha-Beta Pruning Algorithm for Gaming

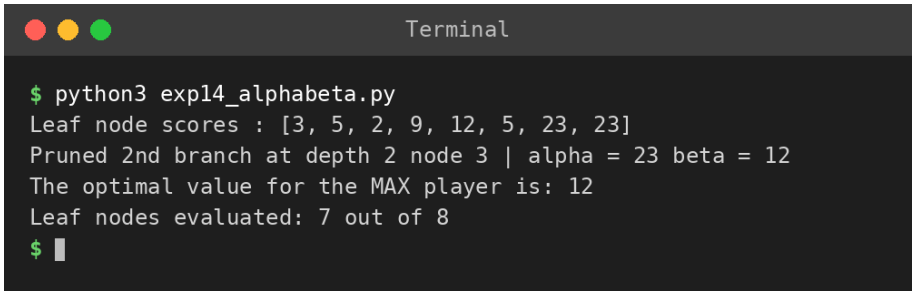
Aim

To write a Python program to implement the Alpha-Beta pruning algorithm for gaming in Artificial Intelligence.

Programming Code:

```
import math

leaf_count = 0

def alpha_beta(depth, node_index, is_max, scores, height, alpha, beta):
    global leaf_count

    # Leaf node reached - return its score
    if depth == height:
        leaf_count += 1
        return scores[node_index]

    if is_max:
        best = -math.inf
        for i in range(2):
            value = alpha_beta(depth + 1, node_index * 2 + i, False,
                               scores, height, alpha, beta)
            best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                if i == 0:
                    print("Pruned 2nd branch at depth", depth, "node", node_index,
                          "| alpha =", alpha, "beta =", beta)
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            value = alpha_beta(depth + 1, node_index * 2 + i, True,
                               scores, height, alpha, beta)
            best = min(best, value)
            beta = min(beta, best)
            if beta <= alpha:
                if i == 0:
                    print("Pruned 2nd branch at depth", depth, "node", node_index,
                          "| alpha =", alpha, "beta =", beta)
                break
        return best


scores = [3, 5, 2, 9, 12, 5, 23, 23]
height = int(math.log2(len(scores)))

print("Leaf node scores :", scores)
result = alpha_beta(0, 0, True, scores, height, -math.inf, math.inf)
print("The optimal value for the MAX player is:", result)
print("Leaf nodes evaluated:", leaf_count, "out of", len(scores))
```

OUTPUT:



Expected Output

- The leaf node scores [3, 5, 2, 9, 12, 5, 23, 23] are printed first.
- One pruning message is displayed: the second branch at depth 2, node 3 is skipped because alpha (23) is greater than or equal to beta (12).
- The optimal value for the MAX player is 12, the same value obtained by the Minimax algorithm.
- Only 7 of the 8 leaf nodes are evaluated, which shows that pruning reduces the search without changing the answer.

Result

Thus, the Alpha-Beta pruning algorithm was implemented successfully in Python. It gives the same result as Minimax but skips branches that cannot affect the final decision, so fewer nodes are evaluated.